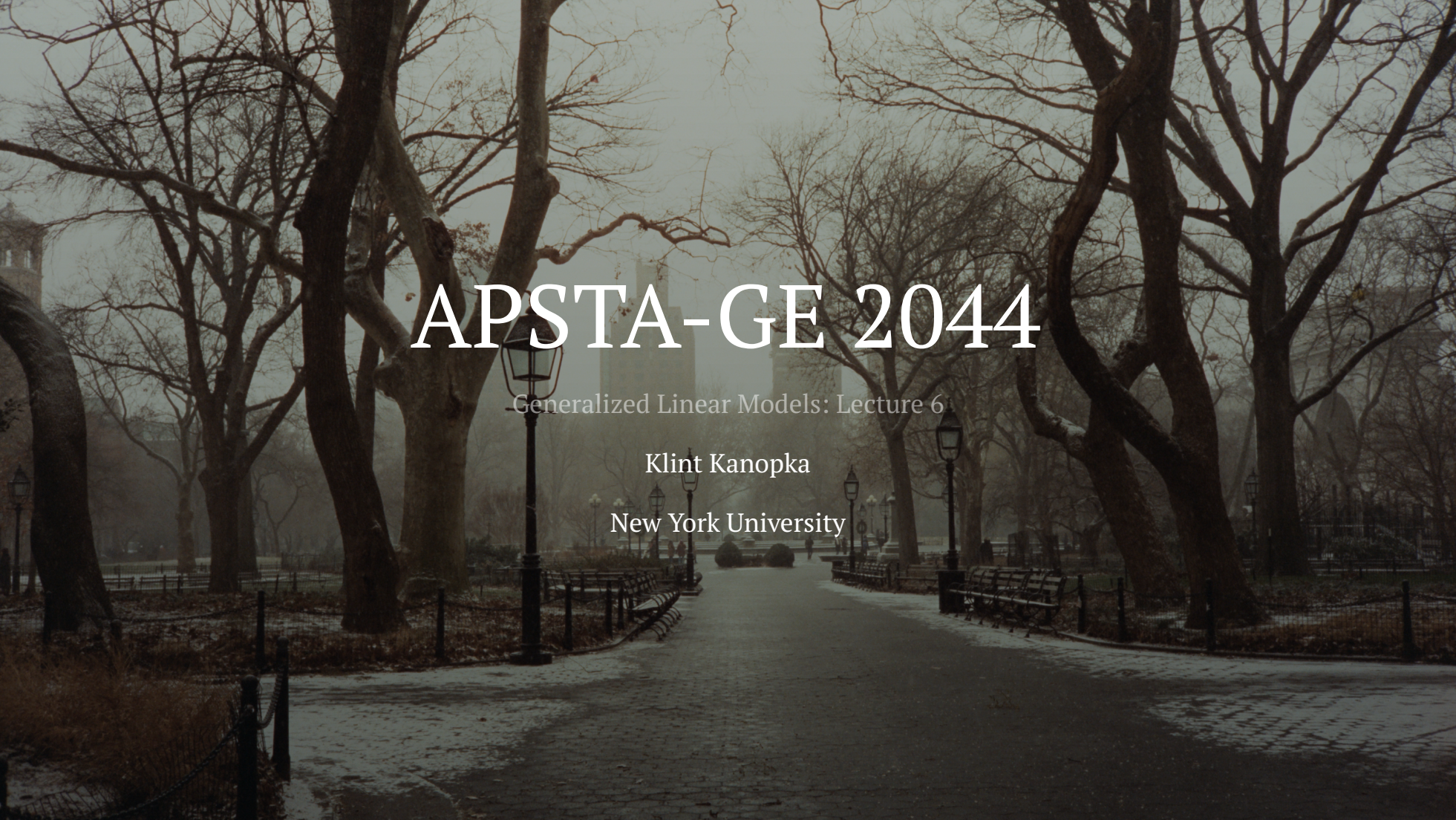




APSTA-GE 2044

Generalized Linear Models: Lecture 6

Klint Kanopka

New York University

Table of Contents

1. Generalized Linear Models - Week 6

1. Table of Contents

2. Course Evaluations

2. Multilevel Model Loose Ends

1. Model Identification

2. Model Convergence

3. Generalized Additive Models

1. Motivating Idea

2. Generalized Additive Models in Practice

3. Visualizing Model Response

4. Wrap Up

Course Evaluations

- As pre-tenure faculty, having these filled out is helpful for me!
- Take 20 minutes to fill out:
 - My personal course feedback form at: <https://forms.gle/fKhGuvnT7HJDyNPW7>
 - The official course evaluations at: <https://coursefeedback.nyu.edu/nyu/>

Multilevel Model Loose Ends

Model Identification

- When I originally wrote and released PS3, it had a *huge* error in it
- What *actually* went wrong?
 - I asked students to add a random slope on minutes of instruction by classroom
 - *Why was this actually a problem?*
 - Minutes of instruction was measured at the classroom level and had no variation within classrooms
 - What does this do?
 - Prevents the model from being *identified*
- If a model is *identified*, it means that there is a unique solution for the estimated parameters
- So what prevents a model from being identified?
 - Too many parameters and too little data!
 - Insufficient variation in the data!
 - Bad parameterizations

Model Identification

- So why was the random slopes model in PS3 not identified?
- Let's start with a simplified equation:
 - $Y_{ij} = \beta_0 + \beta_{0j} + \beta_1 X_j + \varepsilon_{ij}$
 - This works fine—we get an overall intercept, an overall slope, and one intercept per group
 - $Y_{ij} = \beta_0 + \beta_{0j} + \beta_1 X_j + \beta_{1j} X_j + \varepsilon_{ij}$
 - Now why does this break if we add a slope for each group?
 - X_j doesn't vary within groups, so the product $\beta_{1j} X_j$ also doesn't vary within groups
 - This makes it act just like an intercept, so it's competing with β_{0j} for information
 - $Y_j^* = \beta_{0j} + \beta_{1j} X_j$
 - For a given total, Y_j^* , there are an infinite number of choices for β_{0j} , β_{1j} , so the model doesn't have a unique solution

Model Convergence

- When a model *converges*, it means that the software has settled on estimates for all of the parameters
- What causes a model to fail to converge?
 - The model isn't identified
 - This means there isn't a solution
 - There's a problem with the data that prevents it from having a solution
 - The software doesn't find a solution fast enough
- What can you do about it?
 - Let `lmer()` use the log-likelihood criterion for optimization by passing `REML=FALSE`
 - Fit a simpler model
 - Fiddle with estimation controls by passing arguments to `control=lmerControl()`

Final MLM Thoughts

- Multilevel models are really complex
 - A full semester isn't even enough to *really* understand them!
 - But hopefully you have a sense of when they may be useful in your own work and now know what questions to ask
- Keep multilevel models in mind for nested data and longitudinal data
- If you'd like to learn more, check out **APSTA-GE 2356: Practicum in Applied Statistics: Multi-Level Models**

Generalized Additive Models

Motivating Idea

- You want to run the regression:
 - $Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i$
 - You don't think the relationship between X_i , Y_i is linear
 - What can you do?
 - Polynomial transformations
 - ML specifications
 - Something else?
- What about if you have three variables?
 - $Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \varepsilon_i$
- Generalized Additive Models (GAMs) allow you to replace coefficients with *smooth functions* of variables that best fit your data
 - $Y_i = \beta_0 + f_1(X_{1i}) + f_2(X_{2i}) + f_3(X_{3i}) + \varepsilon_i$
 - The model estimates each $f_k(\cdot)$ to best predict the data

Generalized Additive Models in Practice

- The library that holds all the functions you'll need for GAMs is `mgcv`
- The main workhorse functions are going to be `gam()` and `s()`
 - `gam()` takes a formula, data, and an optional `family=` argument, so it handles GLMs natively
 - `s()` is a "smoother" function that fits a flexible curve to your data using *splines*
 - There are a *bunch* of arguments for `s()`, but today we'll use `k`
 - Higher values of `k` allow for more flexible functions
- Usage examples:
 - `gam(y ~ x + z, data = d)` fits a standard linear regression, estimating coefficients for x , z
 - `gam(y ~ s(x) + z, data = d)` fits a GAM with splines on x and a coefficient on z
 - `gam(y ~ s(x) + s(z), data = d)` fits a GAM with (separate) spline functions on x , z
 - `gam(y ~ s(x, k=3) + s(z, k=5), data = d)` fits a GAM with (separate) spline functions on x , z where the fit on z is more flexible (wiggly) than the fit on x

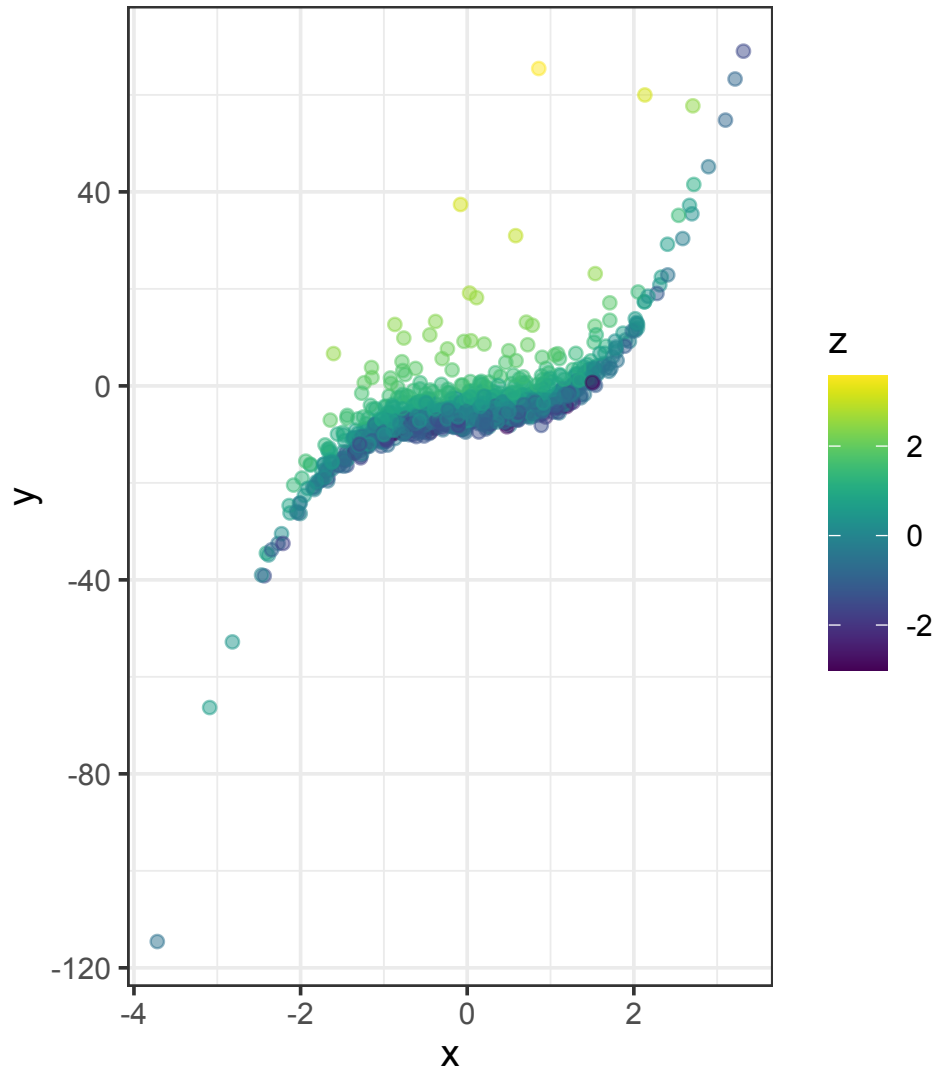
Simulating Some Data

- Let's simulate something that we'd expect a linear regression to do a bad job with:

```
1  set.seed(777)
2
3  N <- 1000
4
5  d <- data.frame(
6    x = rnorm(N),
7    z = rnorm(N)
8  )
9
10 d$y <- -8 + 2 * d$x^3 + d$x + 2 * exp(d$z) + rnorm(N)
```

What does the DGP look like?

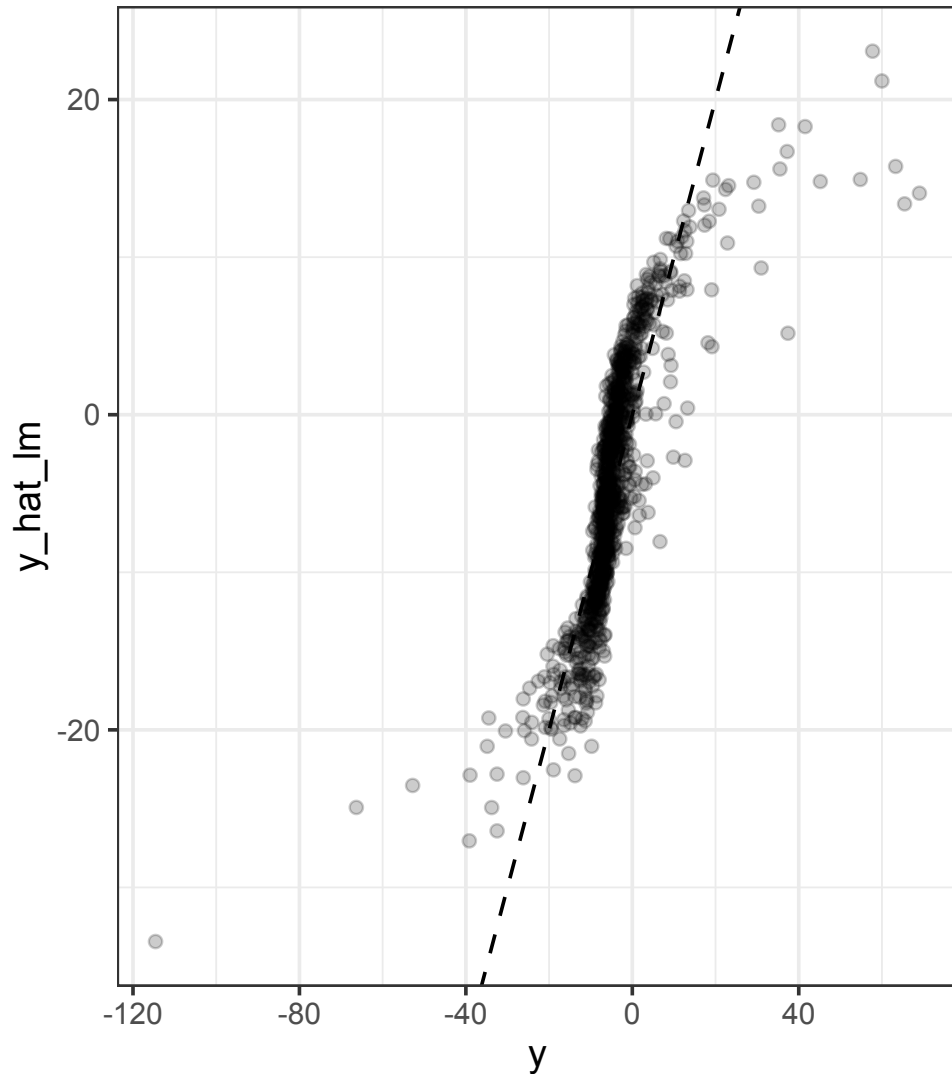
```
1 ggplot(d, aes(x = x, y = y, color = z)) +  
2   geom_point(alpha=0.5) +  
3   scale_color_viridis() +  
4   theme_bw()
```



How well does a linear model predict y ?

```
1 m <- lm(y ~ x + z, data = d)
2
3 d$y_hat_lm <- predict(m)
4
5 ggplot(d, aes(x = y, y = y_hat_lm)) +
6   geom_point(alpha=0.2) +
7   geom_abline(aes(intercept = 0,
8                   slope = 1),
9               lty = 2) +
10  theme_bw()
```

- Not great in the tails!



Looking at GAM Output

- Let's start with just an OLS using the `gam()` function:

```
1 m1 <- gam(y ~ x + z, data = d)
2 summary(m1)
3
4 Family: gaussian
5 Link function: identity
6
7 Formula:
8 y ~ x + z
9
10 Parametric coefficients:
11             Estimate Std. Error t value Pr(>|t|)
12 (Intercept)  -4.6969      0.2049  -22.93  <2e-16 ***
13 x              7.2214      0.2057   35.10  <2e-16 ***
14 z              3.3351      0.2110   15.80  <2e-16 ***
15 ---
16 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
17
18
19 R-sq.(adj) = 0.602   Deviance explained = 60.3%
20 GCV = 42.087   Scale est. = 41.96      n = 1000
```

Looking at More GAM Output

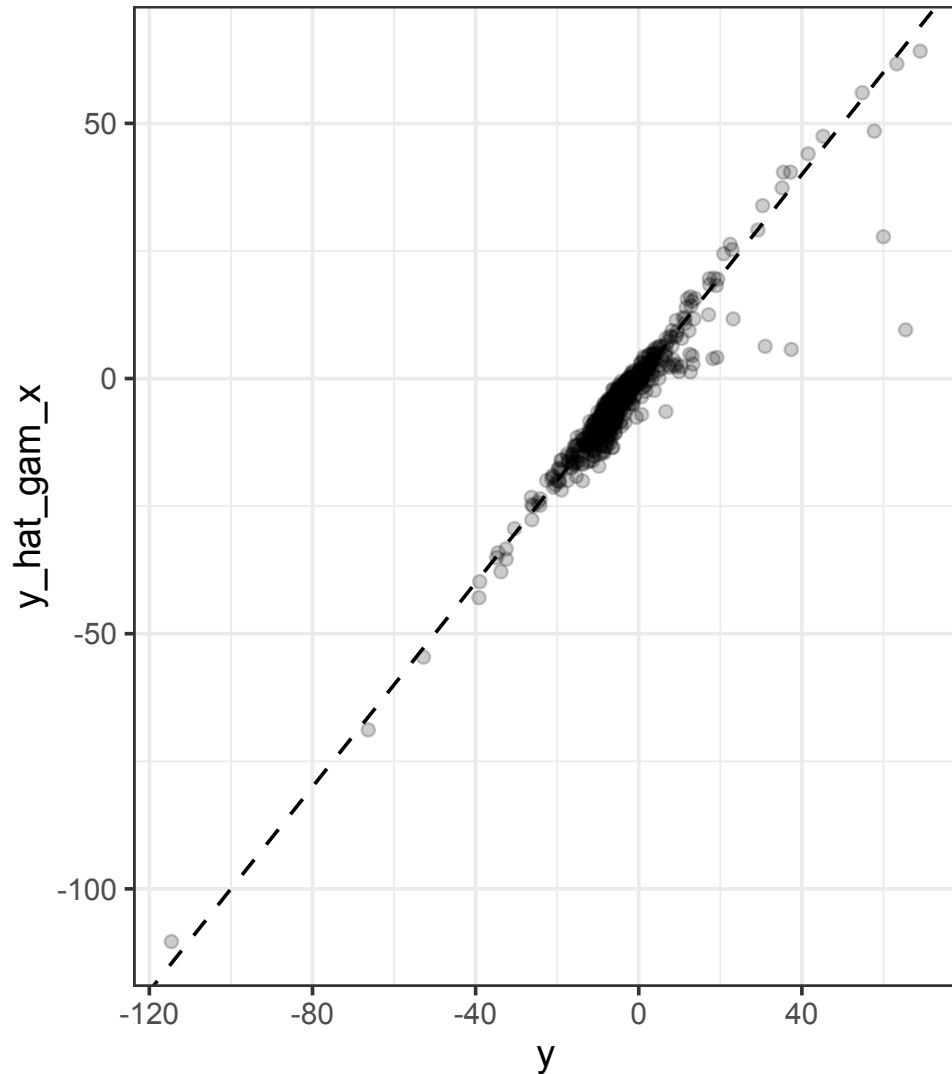
- Now let's add a smoothing function to x :

```
1 m2 <- gam(y ~ s(x) + z, data = d)
2 summary(m2)
3
4 Formula:
5 y ~ s(x) + z
6
7 Parametric coefficients:
8             Estimate Std. Error t value Pr(>|t|)
9 (Intercept) -4.6833      0.1050  -44.61  <2e-16 ***
10 z           3.3986      0.1087   31.28  <2e-16 ***
11 ---
12 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
13
14 Approximate significance of smooth terms:
15             edf Ref.df      F p-value
16 s(x)  8.685   8.972 834.4  <2e-16 ***
17 ---
18 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
19
20 R-sq.(adj) = 0.895   Deviance explained = 89.6%
21 GCV = 11.138   Scale est. = 11.019     n = 1000
```

How well does this
GAM with a smoother
on x predict y ?

```
1 d$y_hat_gam_x <- predict(m2)
2
3 ggplot(d, aes(x = y, y = y_hat_gam_x)) +
4   geom_point(alpha = 0.2) +
5   geom_abline(aes(intercept = 0,
6                   slope = 1),
7               lty = 2) +
8   theme_bw()
```

- Way better!



Looking at More GAM Output

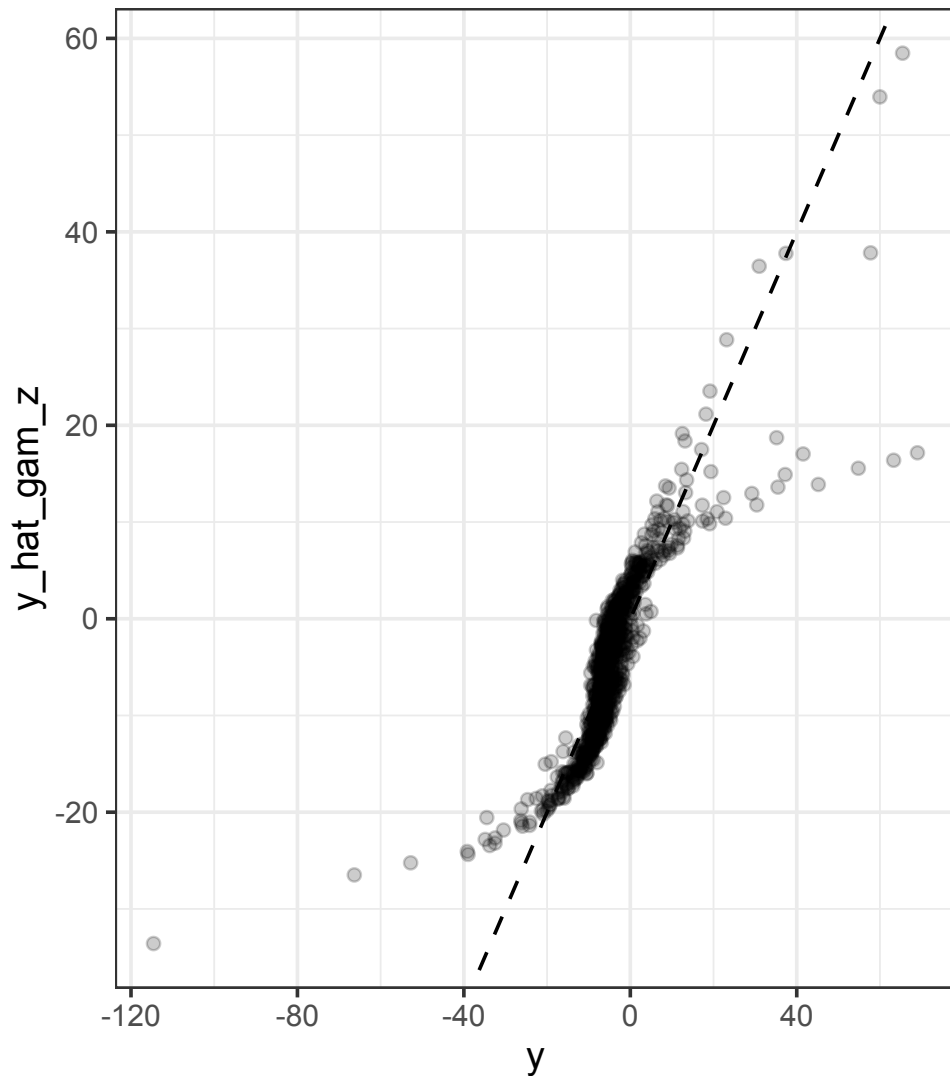
- Now let's add a smoothing function to z :

```
1 m3 <- gam(y ~ x + s(z), data = d)
2 summary(m3)
3
4 Formula:
5 y ~ x + s(z)
6
7 Parametric coefficients:
8             Estimate Std. Error t value Pr(>|t|)
9 (Intercept)  -4.6624      0.1753  -26.59  <2e-16 ***
10 x             7.1789      0.1768   40.61  <2e-16 ***
11 ---
12 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
13
14 Approximate significance of smooth terms:
15             edf Ref.df      F p-value
16 s(z) 6.921  8.036 87.43  <2e-16 ***
17 ---
18 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
19
20 R-sq.(adj) = 0.708   Deviance explained = 71.1%
21 GCV = 31.018   Scale est. = 30.741      n = 1000
```

How well does this GAM with a smoother on z predict y ?

```
1 d$y_hat_gam_z <- predict(m3)
2
3 ggplot(d, aes(x = y, y = y_hat_gam_z)) +
4   geom_point(alpha = 0.2) +
5   geom_abline(aes(intercept = 0,
6                   slope = 1),
7               lty = 2) +
8   theme_bw()
```

- Worse than the smoother on x , but better than no smoothers at all!
- Turns out the nonlinearity in x is more important than the nonlinearity in z



Looking at Even More GAM Output

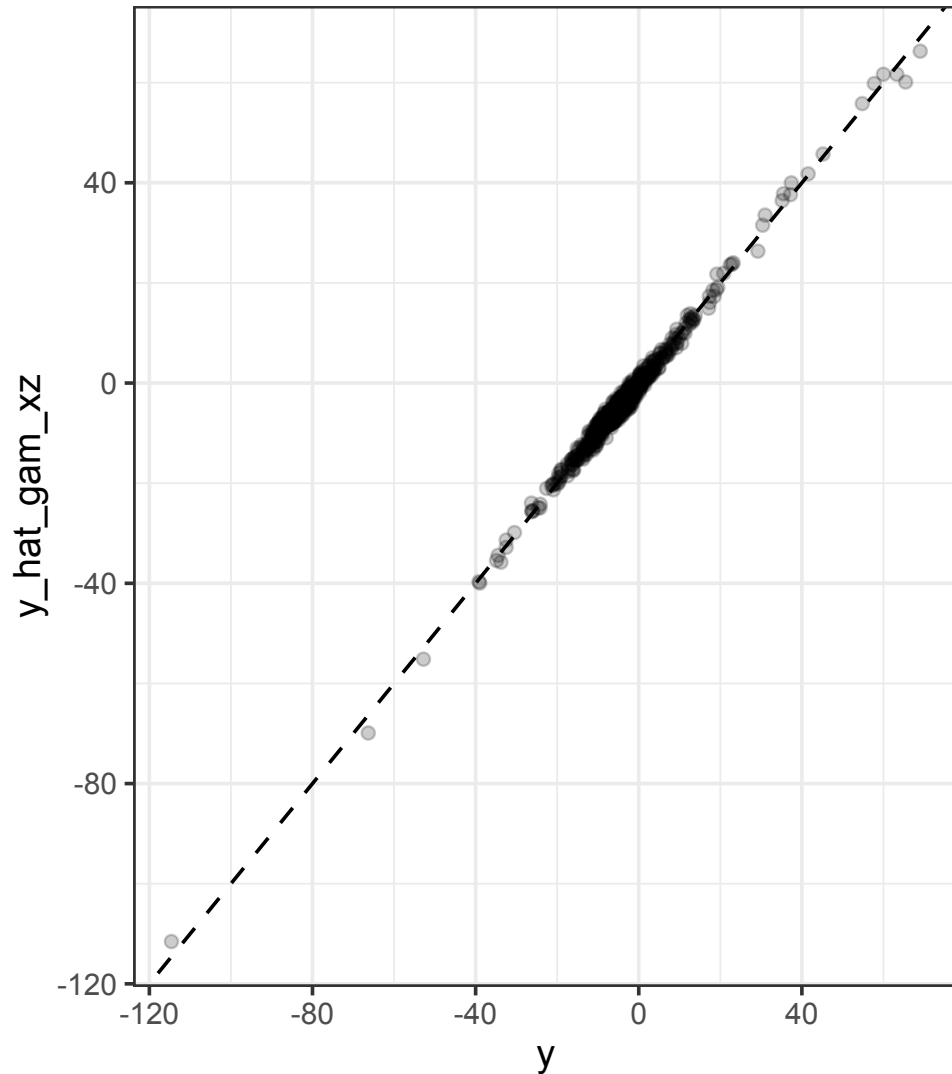
- Now let's add a smoothing function to x and z :

```
1  m4 <- gam(y ~ s(x) + s(z), data = d)
2  summary(m4)
3
4  Formula:
5  y ~ s(x) + s(z)
6
7  Parametric coefficients:
8      Estimate Std. Error t value Pr(>|t|)
9  (Intercept) -4.64823    0.03312  -140.4   <2e-16 ***
10 ---
11 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
12
13 Approximate significance of smooth terms:
14      edf Ref.df    F p-value
15 s(x)  8.978  9.000 8092  <2e-16 ***
16 s(z)  8.932  8.999 2087  <2e-16 ***
17 ---
18 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
19
20 R-sq.(adj) =  0.99    Deviance explained =  99%
21 GCV = 1.1179    Scale est. = 1.0968      n = 1000
```

How well does this GAM with a smoother on x and z predict y ?

```
1 d$y_hat_gam_xz <- predict(m4)
2
3 ggplot(d, aes(x = y, y = y_hat_gam_xz)) +
4   geom_point(alpha = 0.2) +
5   geom_abline(aes(intercept = 0,
6                   slope = 1),
7               lty = 2) +
8   theme_bw()
```

- Turns out, pretty damn well!



Visualizing Model Response

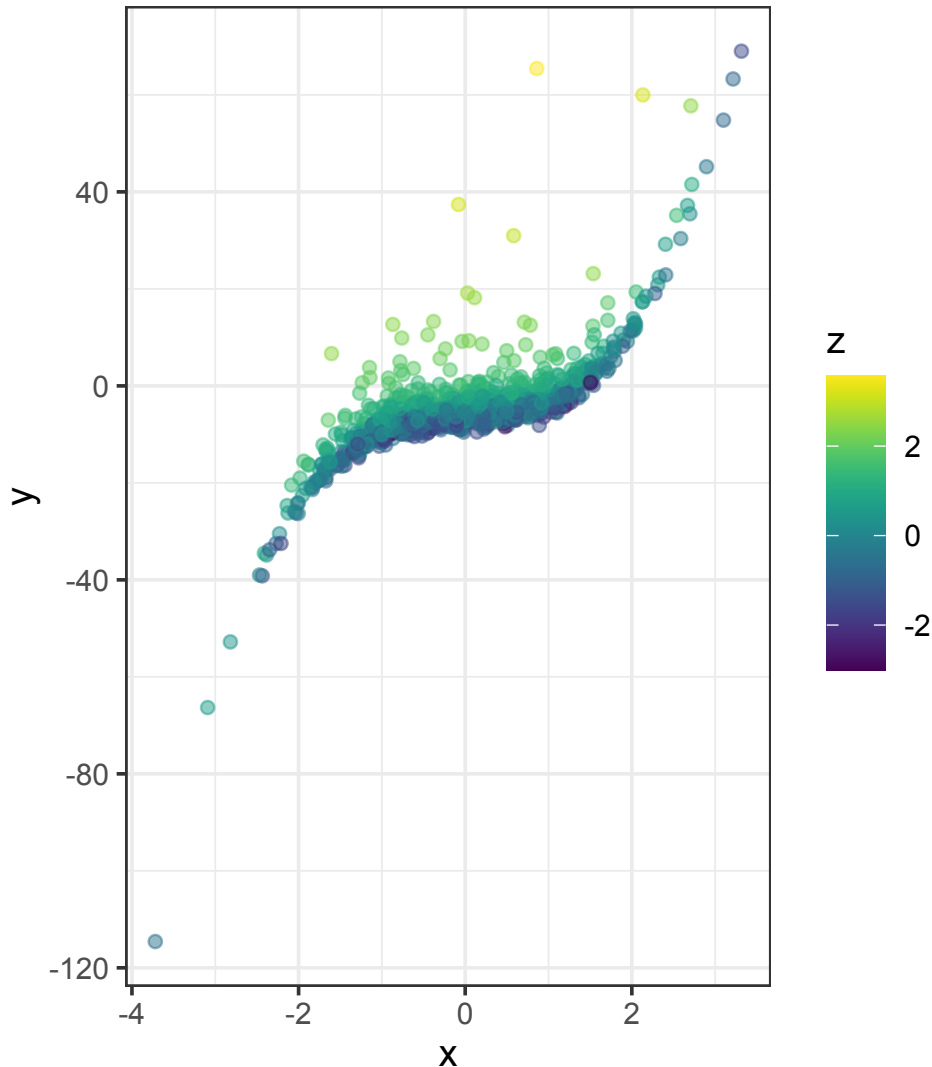
- When you break into really flexible models, it can be hard to understand what your model is doing
- A good technique to help visualize what your model is doing is to pick a focal variable, X , then:
 - Generate a grid of values for X that covers the values in your data
 - Fix the other variables in your dataset to some value (mean, median, some variable of interest, etc)
 - Use the `predict()` function on your model to generate Y values for this new dataset
 - Plot a line for Y vs X
- You can also pick two variables and do the same thing
 - Instead of plotting a line, you plot X_1 vs X_2 and make a heatmap that varies color by Y
 - This can help to understand interactions in complex models

Recall the data:

```
1 ggplot(d, aes(x = x, y = y, color = z)) +  
2   geom_point(alpha=0.5) +  
3   scale_color_viridis() +  
4   theme_bw()
```

- Let's visualize what the GAM learned from this:

```
1 d_grid <- data.frame(x = seq(from = -4, to = 4,  
2                           length.out = 1000))  
3 d_grid$z <- median(d$z)  
4  
5 d_grid$y_pred <- predict(m4, newdata = d_grid)
```

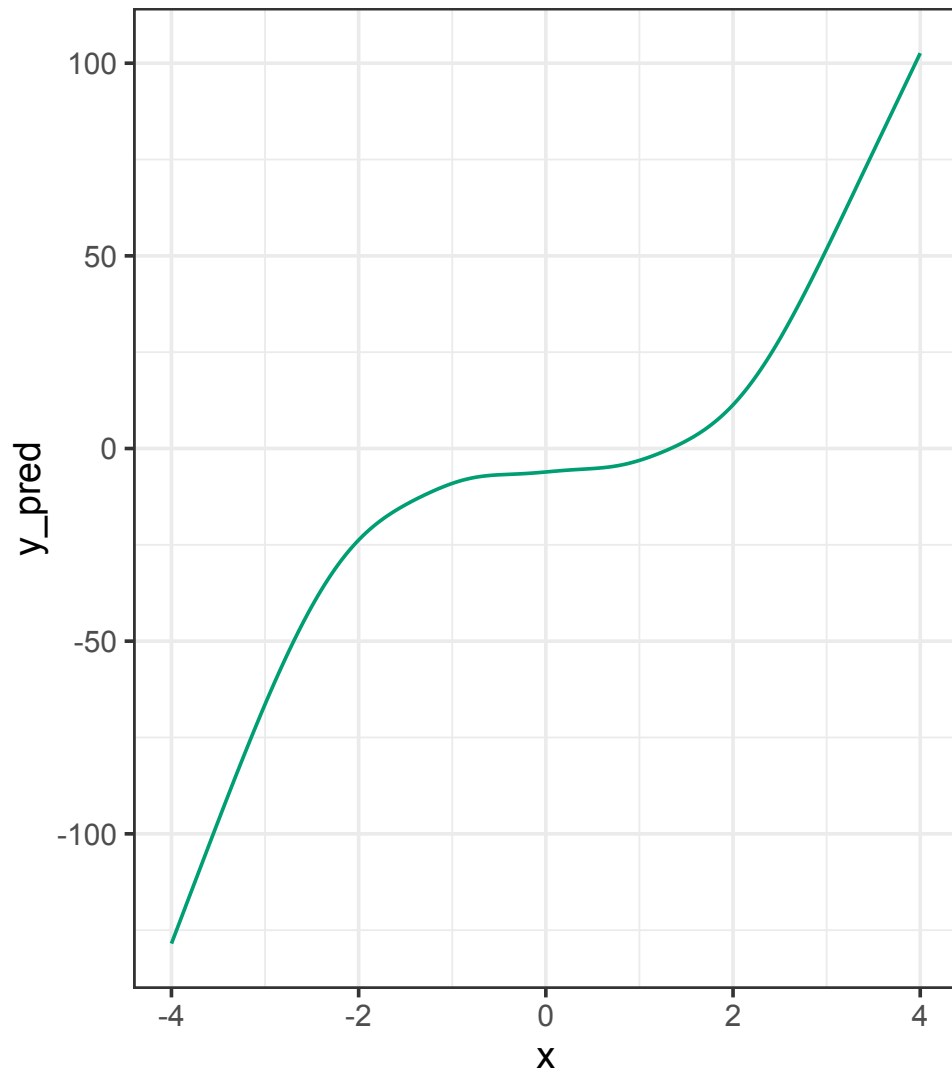


The Response Function:

- Here's how y varies as a function of x when z is held at its median value:

```
1 ggplot(d_grid, aes(x = x, y = y_pred)) +  
2   geom_line(color = okabeito_colors(3)) +  
3   theme_bw()
```

- Looks cubic!

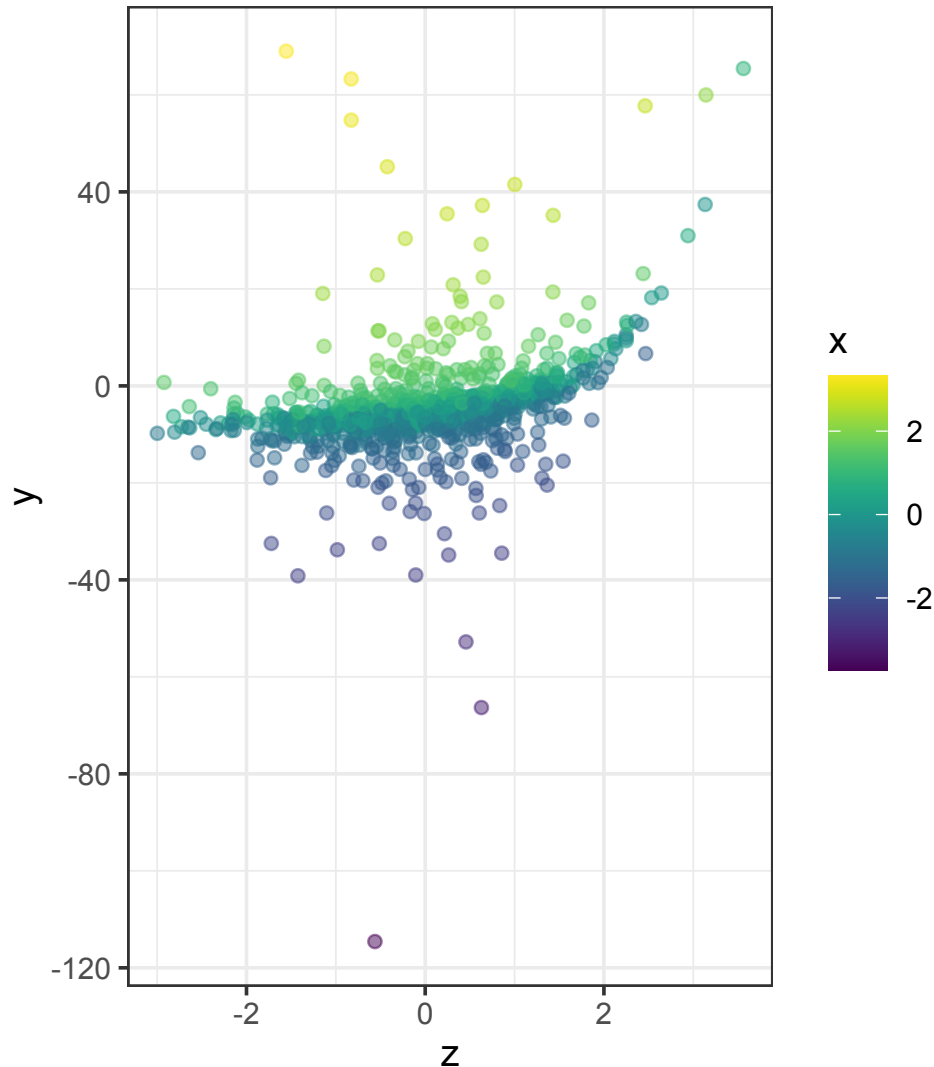


Let's do the same thing for z :

```
1 ggplot(d, aes(x = z, y = y, color = x)) +  
2   geom_point(alpha=0.5) +  
3   scale_color_viridis() +  
4   theme_bw()
```

- Let's visualize what the GAM learned from this:

```
1 d_grid <- data.frame(z = seq(from = -4, to = 4,  
2                           length.out = 1000))  
3 d_grid$x <- median(d$x)  
4  
5 d_grid$y_pred <- predict(m4, newdata = d_grid)
```

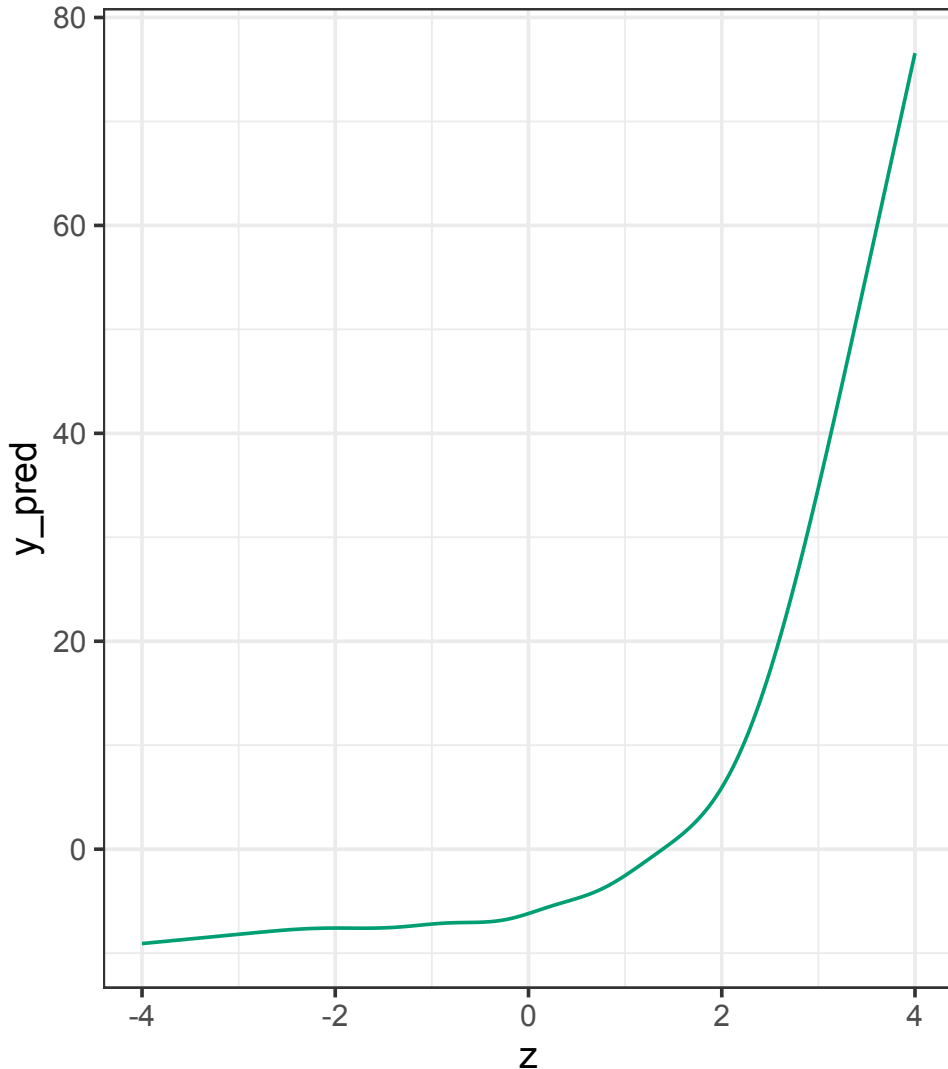


The Response Function:

- Here's how y varies as a function of z when x is held at its median value:

```
1 ggplot(d_grid, aes(z = z, y = y_pred)) +  
2   geom_line(color = okabeito_colors(3)) +  
3   theme_bw()
```

- Looks exponential!



A New DGP

- Just a sum of inverted quadratics!

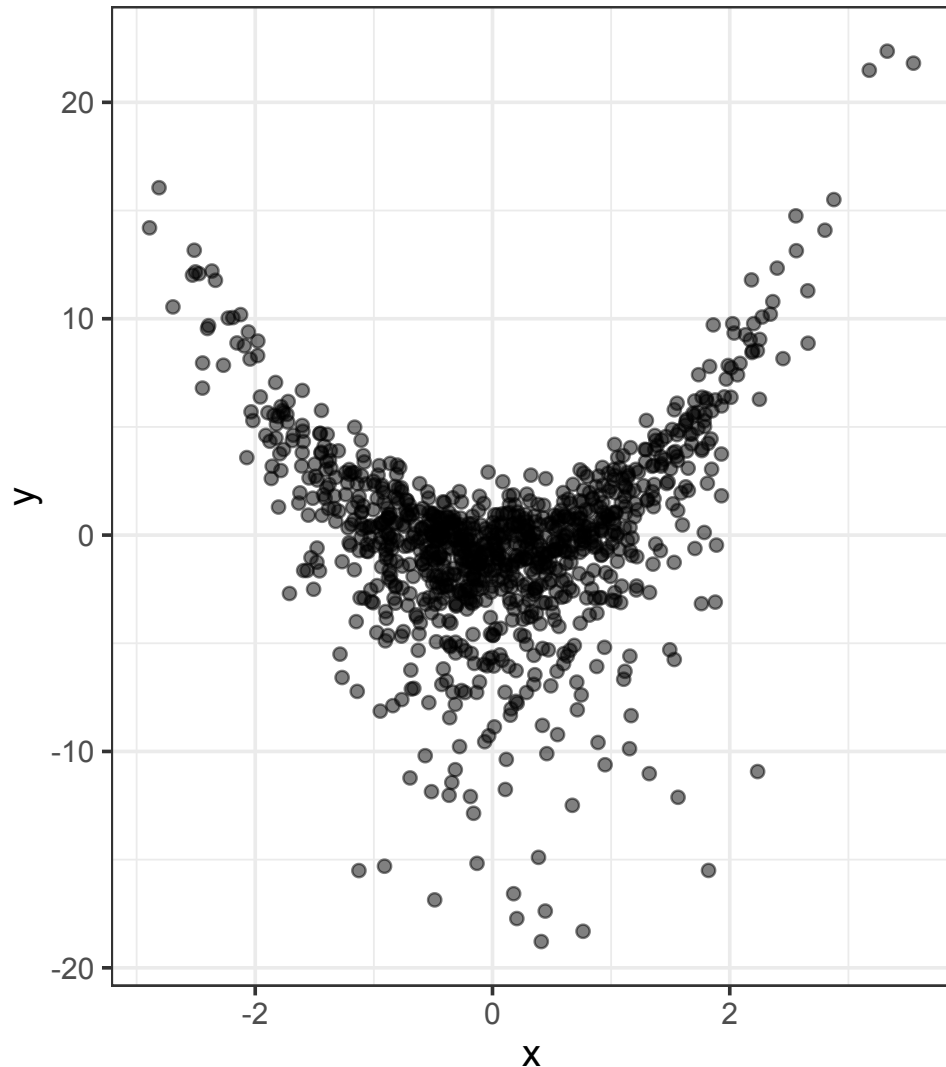
```
1  d_new <- data.frame(  
2    x = rnorm(N),  
3    z = rnorm(N)  
4  )  
5  
6  d_new$y <- 2 * (d_new$x^2) - 2 * (d_new$z^2) + rnorm(N)  
7  m5 <- gam(y ~ s(x) + s(z), data = d_new)
```

Visualizing, starting with x

```
1 ggplot(d_new, aes(x = x, y = y)) +  
2   geom_point(alpha = 0.5) +  
3   theme_bw()
```

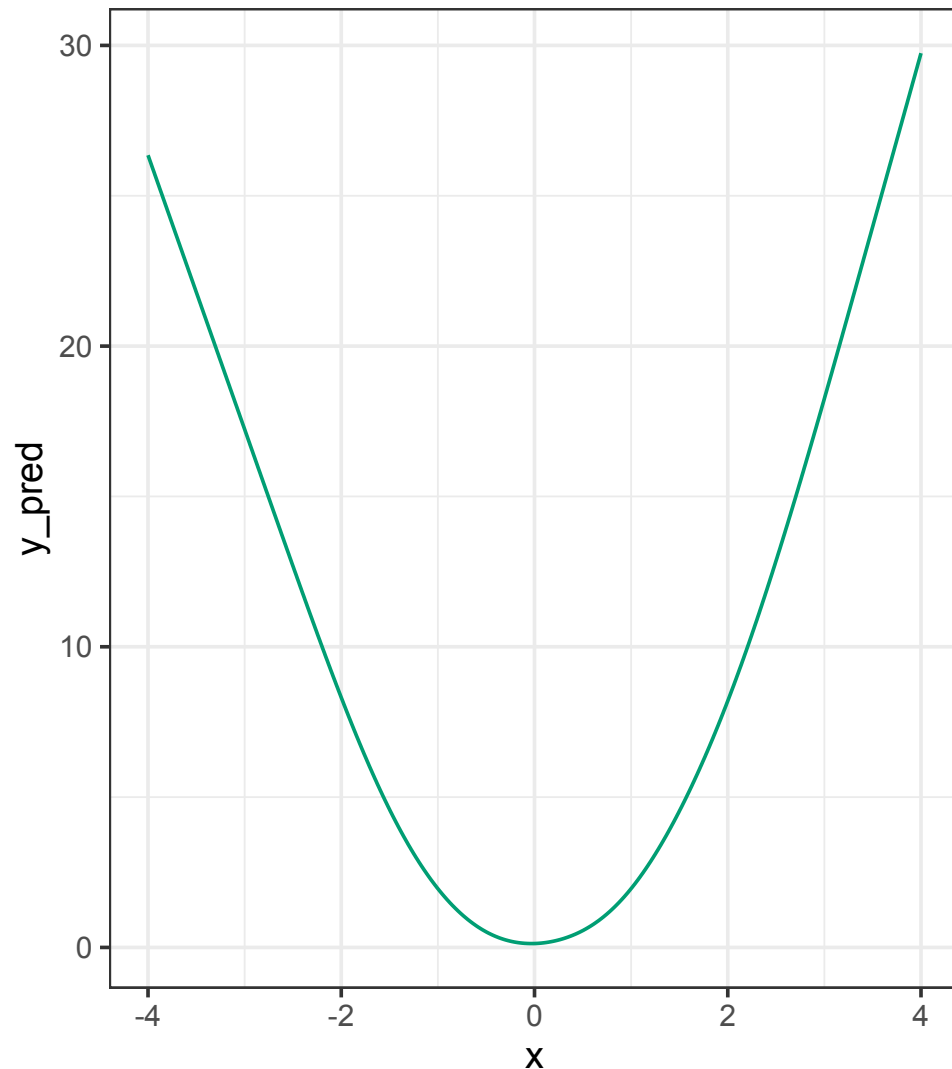
■ As before!

```
1 d_grid <- data.frame(  
2   x = seq(from = -4,  
3         to = 4,  
4         length.out = 1000))  
5 d_grid$z <- median(d$z)  
6  
7 d_grid$y_pred <- predict(m5,  
8                       newdata = d_grid)
```



The GAM response for
 x at the median value
of z

```
1 ggplot(d_grid, aes(x = x, y = y_pred)) +  
2   geom_line(color = okabeito_colors(3)) +  
3   theme_bw()
```

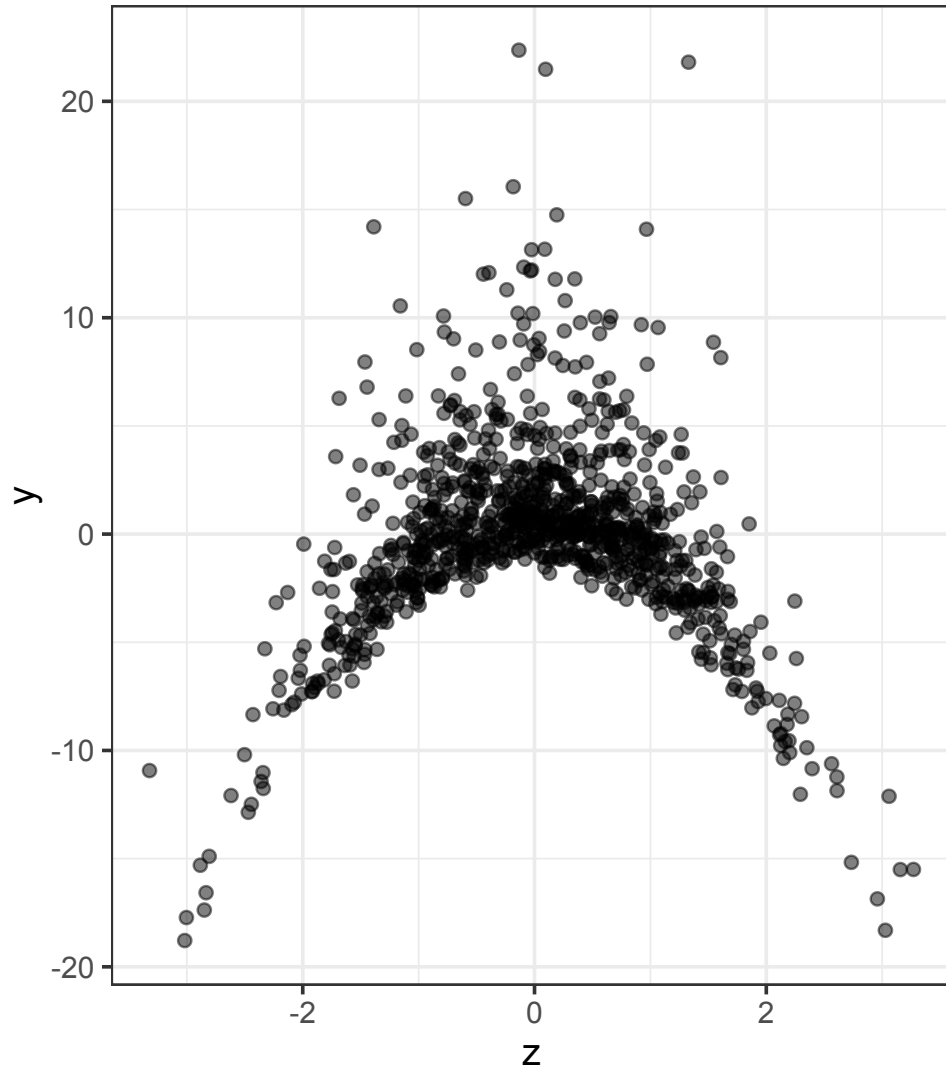


Visualizing for z

```
1 ggplot(d, aes(x = z, y = y)) +  
2   geom_point(alpha = 0.5) +  
3   theme_bw()
```

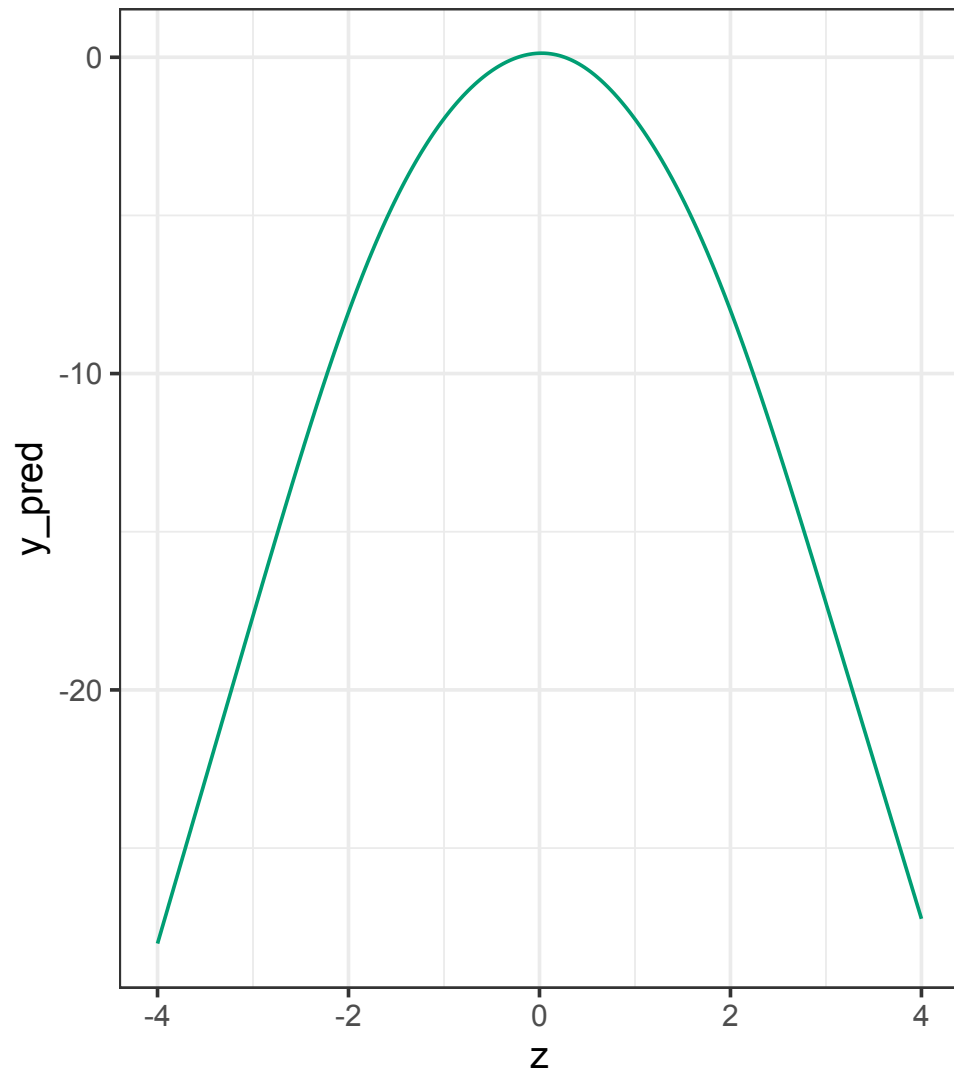
■ One more time!

```
1 d_grid <- data.frame(  
2   z = seq(from = -4,  
3         to = 4,  
4         length.out = 1000))  
5 d_grid$x <- median(d$x)  
6  
7 d_grid$y_pred <- predict(m5,  
8                       newdata = d_grid)
```



The GAM response for
 z at the median value
of x

```
1 ggplot(d_grid, aes(x = z, y = y_pred)) +  
2   geom_line(color = okabeito_colors(3)) +  
3   theme_bw()
```



Visualizing y as a function of x and z

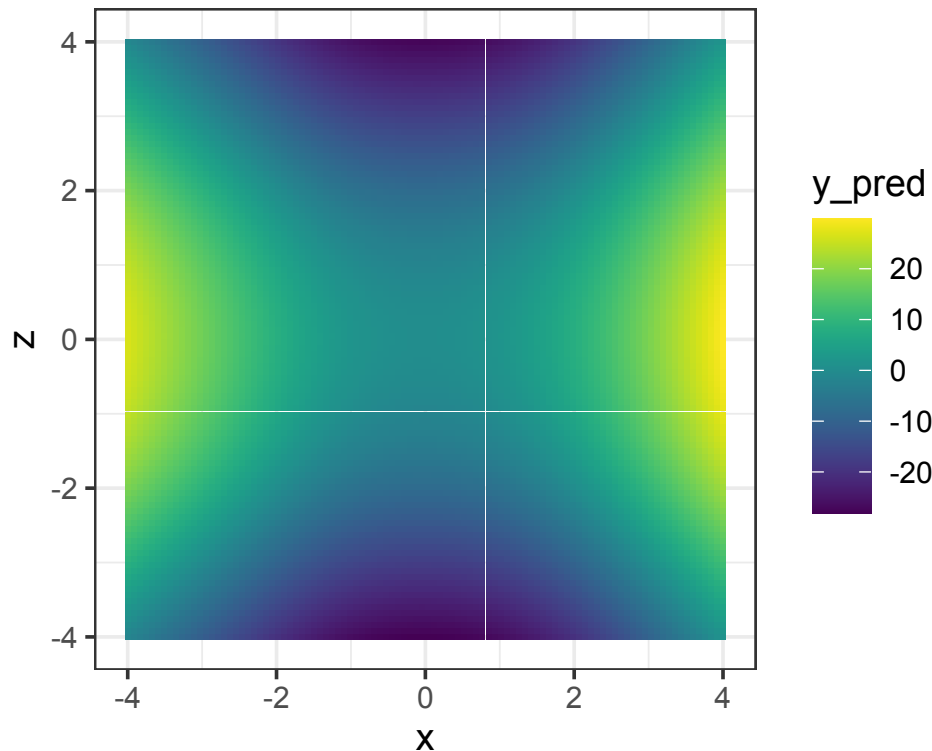
- Here the strategy is to create a grid of x and z values and predict y across them:

```
1 x <- z <- seq(-4, 4, length.out = 100)
2
3 d_grid <- expand.grid(list(x = x, z = z))
4
5 d_grid$y_pred <- predict(m5, newdata = d_grid)
```

The GAM response as a function of both x and

z

```
1  ggplot(d_grid, aes(x = x,  
2                      y = z,  
3                      fill = y_pred)) +  
4    geom_tile() +  
5    coord_equal() +  
6    scale_fill_viridis() +  
7    theme_bw()
```



Wrap Up

Wrap Up

- You should always fit models that match the distribution of your outcome variable
- GLMs let us do that with regressions!
 - Logistic/Probit regressions are good for binary outcomes
 - Poisson, Quasipoisson, and Negative Binomial are good for count variables
 - Gamma and Inverse Gaussian are good for non-negative continuous data
- Multilevel models are good if you have nested data or longitudinal data
 - They let you vary intercepts and slopes by group membership
 - You can only make random effects for variables measured below the group you're interested in
 - GLMs work here
- Generalized Additive Models let you fit flexible functions to variables in your regressions
 - GLMs also work here